

The slide features a large white circle in the center. To the left of the circle, a yellow dashed arc curves upwards. To the right of the circle, a solid orange circle is positioned. The background is a dark, textured grey.

UNIT 3

Constraints, Views and SQL

Contents of Unit 3:

-
- Constraints
 - Types of constraints
 - Integrity constraints
 - Views: Introduction to views
 - Data independence, security,
 - Updates on views
 - Comparison between tables and views
 - SQL: data definition, aggregate function, Null Values, nested sub queries, Joined relations. Triggers.

Constraints



Constraints are the rules that we can apply on the type of data in a table. That is, we can specify the limit on the type of data that can be stored in a particular column in a table using constraints.

Types of constraints

NOT NULL: This constraint tells that we cannot store a null value in a column. That is, if a column is specified as NOT NULL then we will not be able to store null in this particular column anymore.

UNIQUE: This constraint when specified with a column, tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.

PRIMARY KEY: A primary key is a field which can uniquely identify each row in a table. And this constraint is used to specify a field in a table as primary key.

Types of constraints Contd..

FOREIGN KEY: A Foreign key is a field which can uniquely identify each row in another table. And this constraint is used to specify a field as Foreign key.

CHECK: This constraint helps to validate the values of a column to meet a particular condition. That is, it helps to ensure that the value stored in a column meets a specific condition.

DEFAULT: This constraint specifies a default value for the column when no value is specified by the user.

NOT NULL

If we specify a field in a table to be NOT NULL. Then the field will never accept null value. That is, you will be not allowed to insert a new row in the table without specifying any value to this field.

For example, the below query creates a table Employee with the field SALARY as NOT NULL. That is, we are bound to specify values for this field every time we wish to insert a new row.

```
create table emp(eid number(10), esal number(5) not null, eadd char(10));
```

```
insert into emp values(2,15000,null);
```

```
insert into emp values(null,15000,'wadala');
```

```
insert into emp values(2,null,'vashi');// can't execute
```

```
Select * from emp
```

ORA-00947: not enough values

UNIQUE

This constraint helps to uniquely identify each row in the table. i.e. for a particular column, all the rows should have unique values. We can have more than one UNIQUE columns in a table.

For example, the below query creates a table Student where the field CONTACTNO is specified as UNIQUE. i.e, no two employees can have the same phone number. Unique constraint in detail.

```
create table emp3(eid number(10) primary key, ename char(10), contactno number(10) unique);
```

```
insert into emp3 values(3,'rahul',9773494128);
```

```
insert into emp3 values(4,'nita',9773494128); //can't execute
```

```
ORA-00001: unique constraint (SYSTEM.SYS_C004406) violated
```

```
select * from emp3;
```

PRIMARY KEY

Primary Key is a field which uniquely identifies each row in the table. If a field in a table as primary key, then the field will not be able to contain NULL values as well as all the rows should have unique values for this field. So, in other words we can say that this is combination of NOT NULL and UNIQUE constraints.

A table can have only one field as primary key. Below query will create a table named Emp and specifies the field ID as primary key.

```
create table Emp(eid number(10) primary key, esal number(5));
```

```
insert into Emp values(1,25000);
```

```
insert into Emp values(1,5000); // can't execute
```

```
ORA-00001: unique constraint (SYSTEM.SYS_C004404) violated
```


FOREIGN KEY

Foreign Key is a field in a table which uniquely identifies each row of another table. That is, this field points to primary key of another table. This usually creates a kind of link between the tables.

Syntax:

```
create table dept(deptid char(10) primary key, deptname char(10));
```

```
create table emp(eid number(10) primary key, ename char(10), deptid  
references dept(deptid));
```

CHECK

Using the CHECK constraint we can specify a condition for a field, which should be satisfied at the time of entering values for this field.

For example,

1. create table e(eid int primary key,ename char(20),esal int, check(ename like 'p%'));
2. create table stud(rollno int primary key,sname char(20), degree char(10),check(degree in('MBA','MCA','MSC')));

DEFAULT

This constraint is used to provide a default value for the fields. That is, if at the time of entering new records in the table if the user does not specify any value for these fields then the default value will be assigned to them.

For example, the below query will create a table named Student and specify the default value for the field AGE as 18.

```
CREATE TABLE Student  
(  
ID int(6) NOT NULL,  
NAME varchar(10) NOT NULL,  
AGE int DEFAULT 18  
);
```

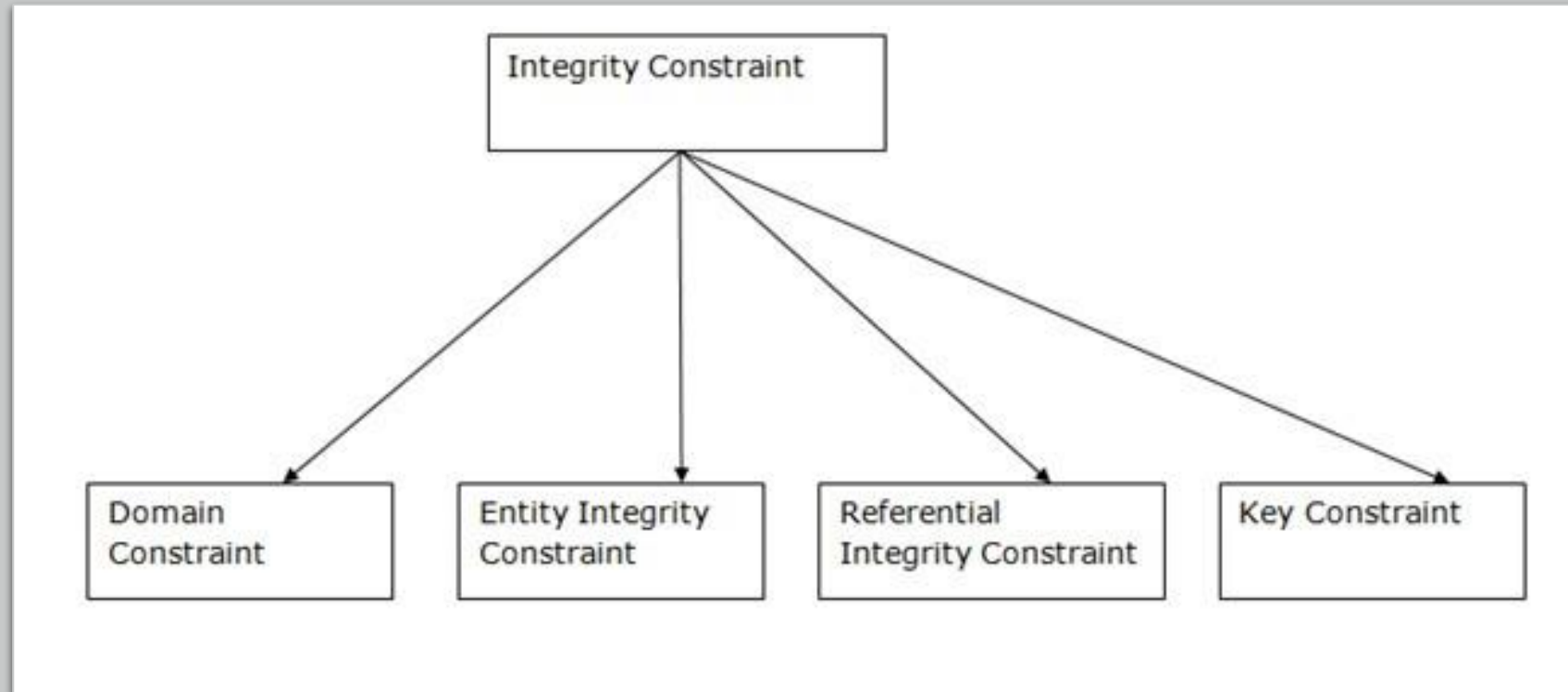
Difference between primary key and foreign key

Primary Key	Foreign Key
Primary key uniquely identify a record in a table	Foreign key is a field in the table that is primary key in another table
Primary key can't accept null values	Foreign key can accept null values
By default, primary key is clustered index and data in the database table is physically organized in the sequence of clustered index	Foreign key does not automatically create an index, clustered or non-clustered. You can manually create an index on foreign key.
We can have only one primary key in a table.	We can have more than one foreign key in a table.

Integrity Constraints

-
- Integrity constraints are set of rules. It is used to maintain the quality of information.
 - Integrity constraints ensure that the data insertion, updating and other processes have to be performed in such a way that data integrity is not affected.
 - Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



Domain constraints

- Domain constraints can be defined as the definition of a set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency etc. The value of attribute must be available in the corresponding domain.
- This can be achieved by using check constraint which is a condition that defines valid data while adding or updating.

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

Entity integrity constraints

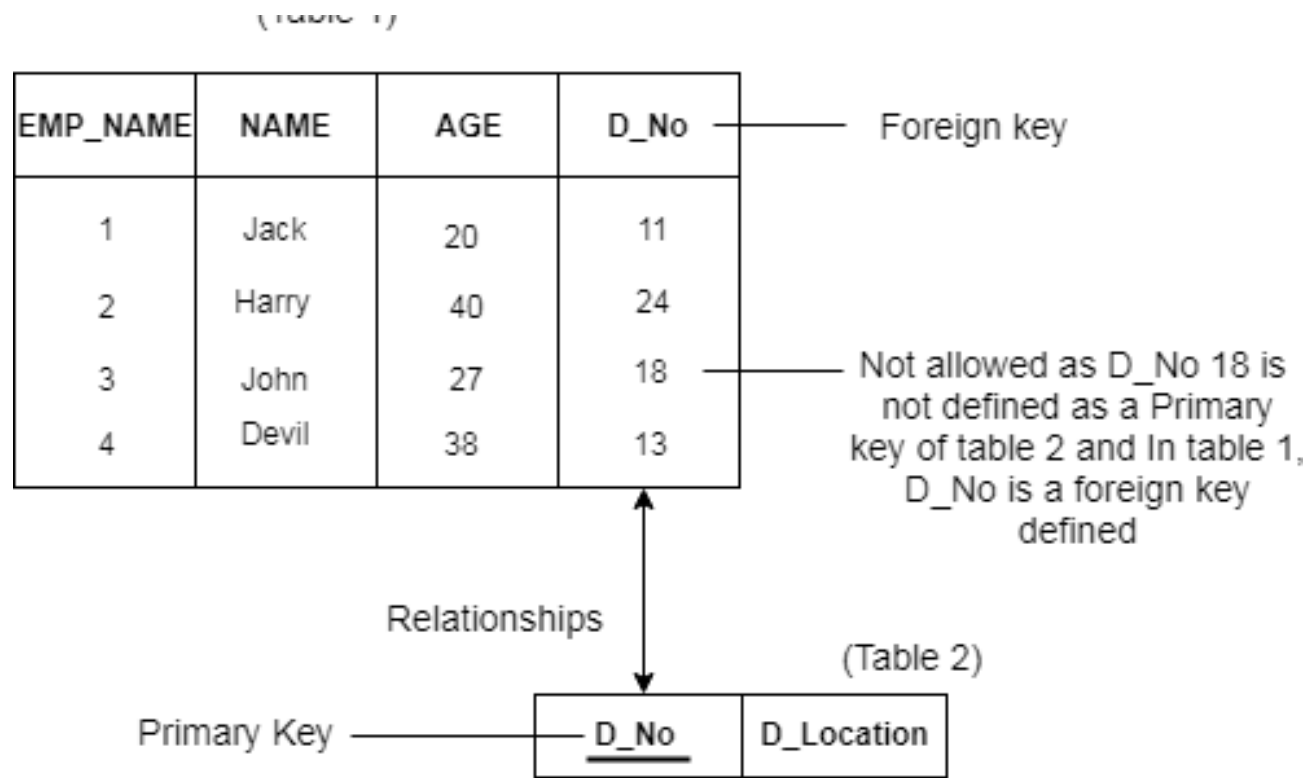
- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in a relation and if the primary key has null value then we can't identify those rows.
- A table can contain null value other than primary key field.

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

Referential Integrity Constraints



- Referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2 then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Key constraints

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

- Keys are the entity set that is used to uniquely identify an entity within its entity set.
- An entity set can have multiple key but out of which one key will be primary key. A primary key can contain only unique and null value in the relational table.



A view is like a virtual table based on the result-set of a SQL statement.



A view can be based on one or more tables or other views. These tables/views are called **base tables**.



A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.



A view takes up no storage space other than for the definition of the view in the data dictionary.

- **Creating the View from a single table.**

Syntax:

```
CREATE VIEW DetailsView AS  
SELECT NAME, ADDRESS  
FROM StudentDetails  
WHERE S_ID < 5;
```

- **To call a view:**

Syntax:

- **SELECT * FROM DetailsView;**

S_ID	NAME	ADDRESS
1	Harsh	Kolkata
2	Ashish	Durgapur
3	Pratik	Delhi
4	Dhanraj	Bihar
5	Ram	Rajasthan

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

Output

```
select * from emp //from table
```

o/p:

1	neha	20000	1
2	neeta	25000	3
3	priya	30000	4
4	raj	35000	2
5	ankita	NULL	NULL

Delete on view

- delete from empdet where
- select * from empdet //from
- output:

eid	ename
1	neha
2	neeta
3	priya
4	raj

- select * from emp //from table
- output:

eid	ename	esal	did
1	neha	20000	1
2	neeta	25000	3
3	priya	30000	4
4	raj	35000	2

Read Only View

- We can create a view with read-only option on the view.
- Syntax to create a view with Read-Only Access

```
CREATE or REPLACE view view_name AS  
SELECT column_name(s)  
FROM table_name  
WHERE condition with read only
```
- The above syntax will create view for read-only purpose. Update or Insert data into read-only view. It will throw error.
- create view emp_det2 as(select eid,ename from emp2)
with read only
- select * from emp_det2
- insert into emp_det2 values(7,'Nita') //can't execute
- update emp_det2 set ename='rahul' where eid=6 //can't execute
- delete from emp_det2 where eid=6 // //can't execute

Difference between table and view

Tables

Hold actual data.

It occupies memory space.

A table can contain its own data.

A record of a table can be inserted, updated and deleted unless it violates any integrity constraints.

You can add integrity constraints to a table.

Views

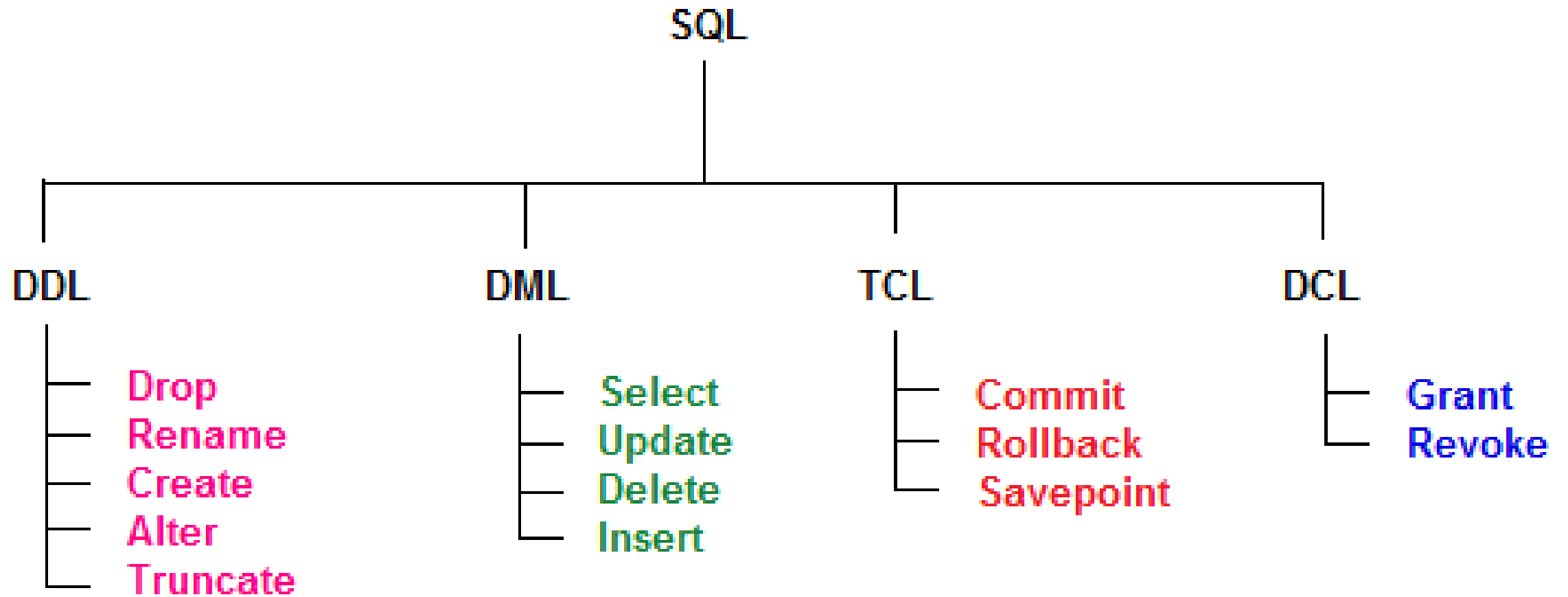
Contains columns which reference the base table for the data.

It does not occupy memory space except a data dictionary entry.

Views contain data from multiple tables.

There are limitations and restrictions to update and delete from views.

You cannot add integrity constraints to a view.



DDL (Data Definition Language)

Data Definition Language is used to define database structure or schema. DDL is also used to specify additional properties of the data. The storage structure and access methods used by the database system by a set of statements in a special type of DDL called a data storage and definition language.

CREATE : to create objects in database

ALTER : alters the structure of database

DROP : delete objects from database

RENAME : rename an objects

TRUNCATE: Delete all objects from database

DML statements are used for managing data within schema objects.

SELECT : retrieve data from the database.

INSERT : insert data into a table.

UPDATE : update existing data within a table.

DELETE : deletes all records from a table.

DML (Data Manipulation Language)



Transaction Control Language commands are used to manage transactions in database. These are used to manage the changes made by DML-statements. It also allows statements to be grouped together into logical transactions.



Examples of TCL commands –



COMMIT : Commit command is used to permanently save any transaction into database.



ROLLBACK : This command restores the database to last committed state. It is also used with savepoint command to jump to a savepoint in a transaction.



SAVEPOINT : Savepoint command is used to temporarily save a transaction so that you can rollback to that point whenever necessary.



A Data Control Language is a syntax like a computer programming language used to control access to data stored in a database (Authorization). It is a component of Structured Query Language (SQL).



Examples of DCL commands :



GRANT : allow specified users to perform specified tasks.



REVOKE : cancel previously granted or denied permissions.

Modifying, Deleting

Adding new column in a table

Syntax: ALTER TABLE table_name ADD column_name datatype(size);

Dropping column from table

Syntax: ALTER TABLE table_name DROP COLUMN column_name;

Modifying column of a table

Syntax: ALTER TABLE table_name MODIFY COLUMN column_name datatype(size);

Deleting all records from table.

Syntax: TRUNCATE TABLE tablename;

Deleting Table

Syntax: drop table tablename;

NULL Value



A field with a NULL value is a field with no value.



If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.



```
SELECT LastName, FirstName, Address FROM Persons WHERE Address IS NULL;
```



```
SELECT LastName, FirstName, Address FROM Persons WHERE Address IS NOT NULL;
```


Pattern Matching

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%_ %'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

contents

Joins

Types of
Joins

SQL Join

SQL Join is used to fetch data from two or more tables, which is joined to appear as single set of data.

It is used for combining column from two or more tables by using values common to both tables.



Types of JOIN

Following are the types of JOIN that we can use in SQL:

1.Cross Join

2.Inner

3.Outer

-----Left

-----Right

----- Full

Cross JOIN or Cartesian Product

- This type of JOIN returns the Cartesian product of rows from the tables in Join.
- It will return a table which consists of records which combines each row from the first table with each row of the second table.
- Cross JOIN Syntax:
-----SELECT column-name-list FROM
table-name1 CROSS JOIN table-name2;

Example of Cross JOIN

class table

- ID NAME
- 1 abhi
- 2 adam
- 4 alex

class_info table

- ID Address
- 1 DELHI
- 2 MUMBAI
- 3 CHENNAI



Cross JOIN query

- `SELECT * FROM class CROSS JOIN class_info;`
- The resultset table will look like,





•	ID	NAME	ID	Address
•	1	abhi	1	DELHI
•	2	adam	1	DELHI
•	4	alex	1	DELHI
•	1	abhi	2	MUMBAI
•	2	adam	2	MUMBAI
•	4	alex	2	MUMBAI
•	1	abhi	3	CHENNAI
•	2	adam	3	CHENNAI
•	4	alex	3	CHENNAI

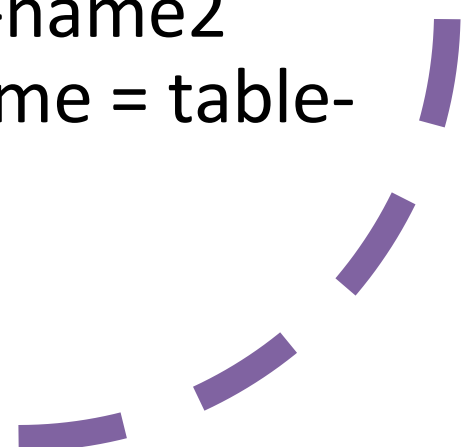
As you can see, this join returns the cross product of all the records present in both the tables.



INNER Join or EQUI Join

- This is a simple JOIN in which the result is based on matched data as per the equality condition specified in the SQL query.
- Inner Join Syntax :

```
-----SELECT column-name-list FROM  
table-name1 INNER JOIN table-name2  
WHERE table-name1.column-name = table-  
name2.column-name;
```



Example of INNER JOIN

Consider a class table

- ID NAME
- 1 abhi
- 2 adam
- 3 alex
- 4 anu

class_info table

- ID Address
- 1 DELHI
- 2 MUMBAI
- 3 CHENNAI



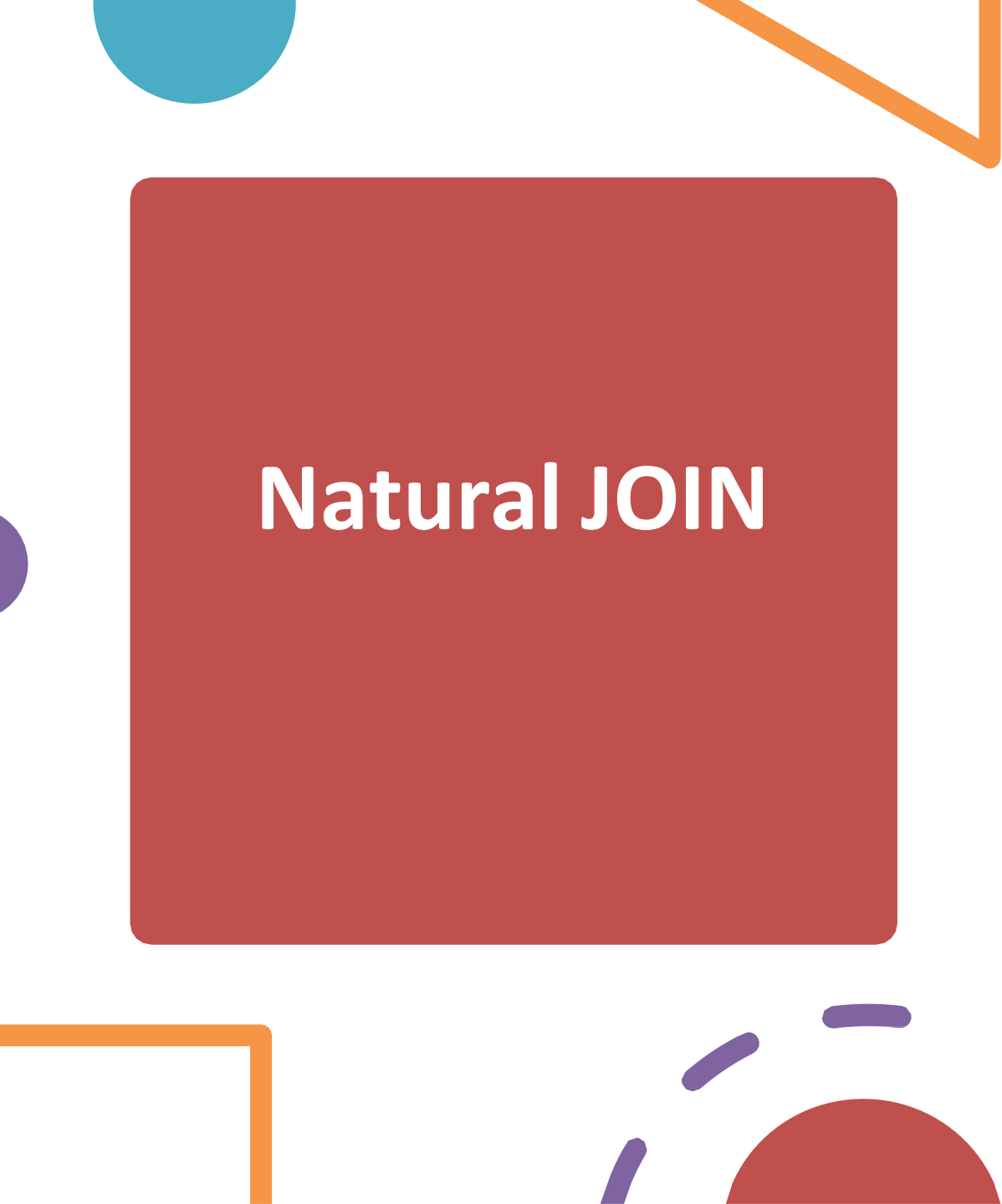
Inner JOIN query

```
--SELECT * from class INNER JOIN  
class_info where class.id =  
class_info.id;
```

- The resultset table will look like,

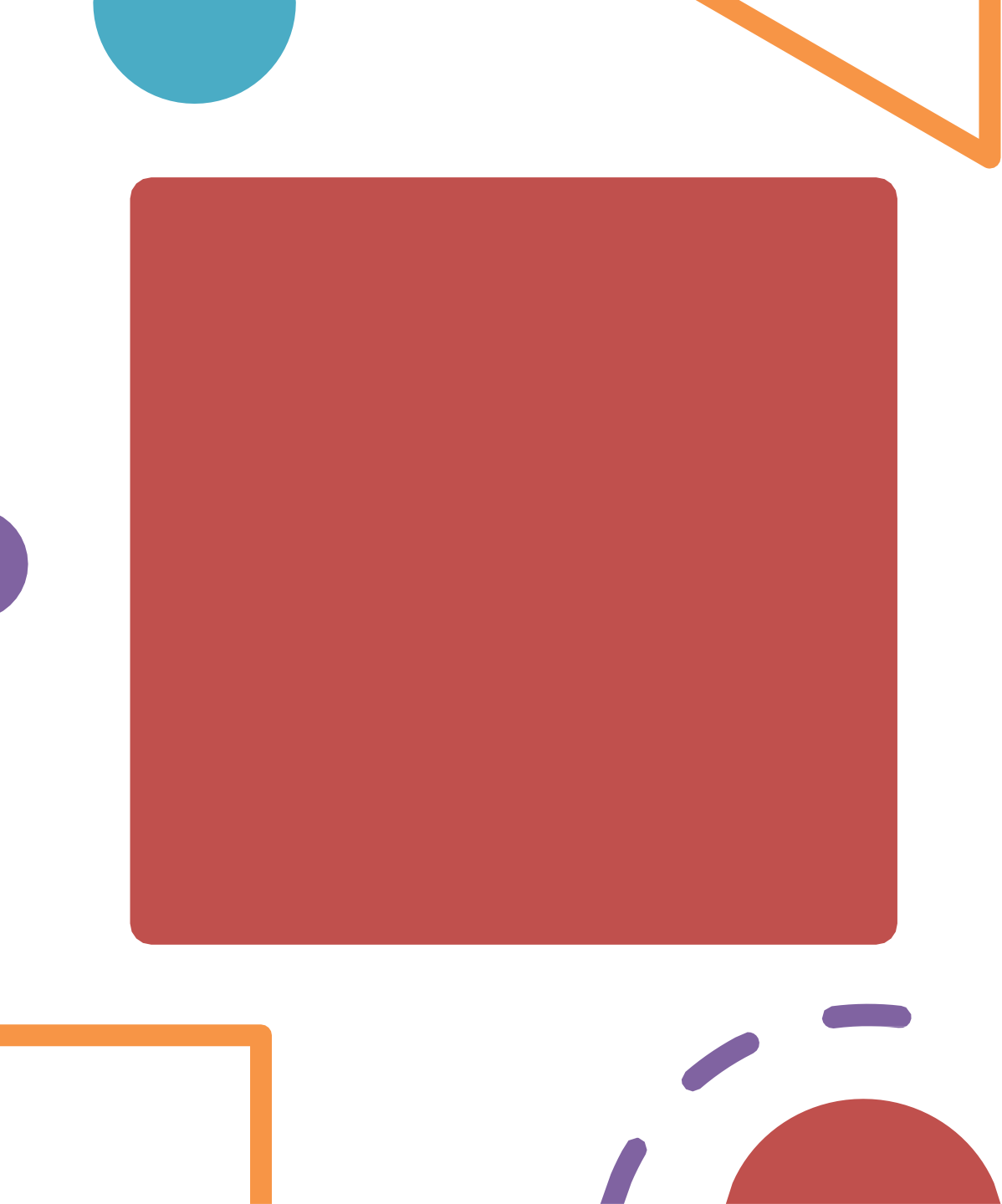
ID	NAME	ID	Address
1	abhi	1	DELHI
2	adam	2	MUMBAI
3	alex	3	CHENNAI





Natural JOIN

- Natural Join is a type of Inner join which is based on column having same name and same datatype present in both the tables to be joined.
- The syntax for Natural Join :
- `SELECT * FROM table-name1 NATURAL JOIN table-name2;`

- 
- Example of Natural JOIN

class table

- ID NAME
- 1 abhi
- 2 adam
- 3 alex
- 4 anu

class_info table

- ID Address
- 1 DELHI
- 2 MUMBAI
- 3 CHENNAI

Natural join query :

- `SELECT * from class NATURAL JOIN class_info;`
- The resultset table will look like,
- | ID | NAME | Address |
|----|------|---------|
| 1 | abhi | DELHI |
| 2 | adam | MUMBAI |
| 3 | alex | CHENNAI |
- In the above example, both the tables being joined have ID column(same name and same datatype), hence the records for which value of ID matches in both the tables will be the result of Natural Join of these two tables.

OUTER JOIN

- Outer Join is based on both matched and unmatched data. Outer Joins subdivide further into,
- Left Outer Join
- Right Outer Join
- Full Outer Join

LEFT Outer Join

The left outer join returns a resultset table with the **matched data** from the two tables and then the remaining rows of the **left** table and null from the **right** table's columns.

Syntax for Left Outer Join is,

```
---SELECT column-name-list FROM table-name1 LEFT OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;
```

To specify a condition, we use the ON keyword with Outer Join.

Example of Left Outer Join

- Here is the class table,
- | ID | NAME |
|----|--------|
| 1 | abhi |
| 2 | adam |
| 3 | alex |
| 4 | anu |
| 5 | ashish |
- and the class_info table,
- | ID | Address |
|----|---------|
| 1 | DELHI |
| 2 | MUMBAI |
| 3 | CHENNAI |
| 7 | NOIDA |
| 8 | PANIPAT |

- Left Outer Join query will be,
- `SELECT * FROM class LEFT OUTER JOIN class_info ON (class.id = class_info.id);`
- The resultset table will look like,
- | ID | NAME | ID | Address |
|----|--------|------|---------|
| 1 | abhi | 1 | DELHI |
| 2 | adam | 2 | MUMBAI |
| 3 | alex | 3 | CHENNAI |
| 4 | anu | null | null |
| 5 | ashish | null | null |

RIGHT Outer Join

- The right outer join returns a resultset table with the **matched data** from the two tables being joined, then the remaining rows of the **right** table and null for the remaining **left** table's columns.
- Syntax for Right Outer Join is,
---SELECT column-name-list FROM table-name1 RIGHT OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;

- Example of Right Outer Join

Once again the class table,

- ID NAME
- 1 abhi
- 2 adam
- 3 alex
- 4 anu
- 5 ashish

and the class_info table,

- ID Address
- 1 DELHI
- 2 MUMBAI
- 3 CHENNAI
- 7 NOIDA
- 8 PANIPAT

- Right Outer Join query will be,
- `SELECT * FROM class RIGHT OUTER JOIN class_info ON (class.id = class_info.id);`
- The resultant table will look like,
- | ID | NAME | ID | Address |
|------|------|----|---------|
| 1 | abhi | 1 | DELHI |
| 2 | adam | 2 | MUMBAI |
| 3 | alex | 3 | CHENNAI |
| null | null | 7 | NOIDA |
| null | null | 8 | PANIPAT |

Full Outer Join

- The full outer join returns a resultset table with the matched data of two table then remaining rows of both left table and then the right table.
- Syntax of Full Outer Join is,
`--SELECT column-name-list FROM table-name1 FULL OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;`

Example of Full outer join:

- The class table,
- ID NAME
- 1 abhi
- 2 adam
- 3 alex
- 4 anu
- 5 ashish

- and the class_info table,
- ID Address
- 1 DELHI
- 2 MUMBAI
- 3 CHENNAI
- 7 NOIDA
- 8 PANIPAT

- Full Outer Join query will be like,
- `SELECT * FROM class FULL OUTER JOIN class_info ON (class.id = class_info.id);`

- The resultset table will look like,

ID	NAME	ID	Address
1	abhi	1	DELHI
2	adam	2	MUMBAI
3	alex	3	CHENNAI
4	anu	null	null
5	ashish	null	null
null	null	7	NOIDA
null	null	8	PANIPAT

SQL Sub Query

A Subquery is a query within another SQL query and embedded within the WHERE clause

A subquery can be placed in a number of SQL clauses like WHERE clause, FROM clause, HAVING clause.

You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements along with the operators like =, <, >, >=, <=, IN, ANY, ALL, BETWEEN, etc.

A subquery is a query within another query. The outer query is known as the main query, and the inner query is known as a subquery.

Subqueries are on the right side of the comparison operator.

A subquery is enclosed in parentheses.

In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

Customers Table

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

1. Subqueries with the Select Statement

- SQL subqueries are most frequently used with the Select statement.

- **Syntax :**

SELECT column_name

FROM table_name

WHERE column_name expression operator

(SELECT column_name from table_name WHERE ...);

- The subquery with a SELECT statement will be:

```
SQL> SELECT *  
FROM CUSTOMERS  
WHERE ID IN (SELECT ID  
FROM CUSTOMERS  
WHERE SALARY > 4500);
```

ID	NAME	AGE	ADDRESS	SALARY
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
7	Muffy	24	Indore	10000.00

2. Subqueries with the INSERT Statement

- SQL subquery can also be used with the Insert statement. In the insert statement, data returned from the subquery is used to insert into another table.
- In the subquery, the selected data can be modified with any of the character, date functions.

- **Syntax:**

```
INSERT INTO table_name (column1, column2, column3....)  
SELECT * FROM table_name WHERE VALUE OPERATOR
```

- **Example**

```
INSERT INTO CUSTOMERS_BKP  
SELECT * FROM CUSTOMERS  
WHERE ID IN (SELECT ID FROM CUSTOMERS);
```

3. Subqueries with the UPDATE Statement

- The subquery of SQL can be used in conjunction with the Update statement. When a subquery is used with the Update statement, then either single or multiple columns in a table can be updated.

- **Syntax**

UPDATE table SET column_name = new_value WHERE VALUE OPERATOR
(SELECT COLUMN_NAME FROM TABLE_NAME WHERE condition);

- **Example**

```
UPDATE CUSTOMERS  
SET SALARY = SALARY * 0.25  
WHERE AGE IN (SELECT AGE FROM  
CUSTOMERS_BKP WHERE AGE >= 27 );
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	125.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	2125.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

4. Subqueries with the DELETE Statement

- The subquery of SQL can be used in conjunction with the Delete statement just like any other statements mentioned above

- **Syntax**

DELETE FROM TABLE_NAME

WHERE VALUE OPERATOR

(SELECT COLUMN_NAME FROM TABLE_NAME WHERE condition);

- **Example**

DELETE FROM CUSTOMERS

WHERE AGE IN (SELECT AGE

FROM CUSTOMERS_BKP

WHERE AGE >= 27);

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Examples of Subqueries

- Display topic for least sequence value
- `Select topic, seq from help where seq=(select min(seq) from help)`
- Display info for highest sequence value
- `Select info, seq from help where seq=(select max(seq) from help)`
- Display topic and sequence from help for less than total number of sequence.
- `Select topic, seq from help where seq<(select count(seq) from help)`
- Display the topic and sequence from help table for sequences having value greater than minimum value.
- `Select topic, seq from help where seq>(select min(seq) from help)`

Employee Table

EMPID	ENAME	JOB	HIREDATE	SALARY
1	Yogesh	AssistantManager	12-JUN-18	50000
2	Chinmay	Analyst	12-JAN-18	5000
3	Dev	Analyst	12-FEB-18	20000
4	Saurabh	Salesman	12-APR-18	5000
5	Aadarsh	Peon	12-MAR-18	500
6	Sahil	Salesman	12-MAY-18	20000
7	Mansoor	Manager	12-JUN-18	50000
8	Yugesh	Assistant	12-JUN-18	30000
9	Sahith	Leader	14-JUN-18	450000
10	Sagar	Projectleader	12-JUN-18	35000

Examples of Subqueries

- Write query to find the salaries for all the employees who have a higher salary than Yogesh.
- Select sal, ename from Empl where salary > (select salary from Empl where ename='Yogesh')
- Display details of an employee whose salary is greater than avg salary of employees who work as salesman.
- Select * from Empl where salary > (select avg(salary) from Empl where job='Salesman')

Examples of Subqueries

- Display the name of the employees whose salary is greater than the employee with empno 10.
- `Select ename from Empl where salary >(select salary from Empl where empno=10);`
- Display empno, ename and job of the employees having salary less than assistant.
- `Select empno, ename, job from Empl where salary < ANY(select sal from Empl where job='Assistant ')`

Advantages and Disadvantages Of Subquery

- **Advantages Of Subquery**

- Subqueries divide the complex query into isolated parts so that a complex query can be broken down into a series of logical steps.
- It is easy to understand and code maintenance is also at ease.
- Subqueries allow you to use the results of another query in the outer query.
- In some cases, subqueries can replace complex joins and unions.

- **Disadvantages of Subquery**

- The optimizer is more mature for joins than for subqueries, so in many cases a statement that uses a subquery can be executed more efficiently if you rewrite it as join.
- We cannot modify a table and select from the same table within a subquery in the same SQL statement.

TRIGGERS

Introduction

- A trigger is a named block of PL/SQL code that is executed or fired automatically in response to a specific data manipulation event.
- A trigger is a database object like a table or view which is stored in the database and is always executed when the event for which it is defined occurs.
- A trigger can be fired before or after an insert, update or delete statement is executed on a table.

Components of Triggers

- **Triggering Event or statement:**

The SQL statements on execution of which the defined trigger code fires is called Triggering SQL statement. For ex. INSERT, UPDATE AND DELETE.

- **Trigger Action**

The PL/SQL block which gets executed when the Triggering SQL statement is executed.

- Trigger Restriction**

We can specify under what condition the trigger should fire.

SYNTAX – CREATE TRIGGER

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE|AFTER}
{DELETE | INSERT | UPDATE [OF col_name1 [,col_name2]...]}
[OR {{DELETE | INSERT | UPDATE [OF col_name1 [,col_name2]...]}....}
ON table_name
[FOR EACH ROW
[WHEN (trigger_condition)]]
pl_sql_block
```

TYPES OF TRIGGERS

Oracle has 13 triggers:

- **Classification of triggers**

The classification of triggers based on whether they fire/execute before or after the triggering statement.

BEFORE TRIGGERS

AFTER TRIGGERS

- **Classification based on whether the trigger fires for each record or table**

STATEMENT LEVEL(TABLE LEVEL)

ROW LEVEL

Classification based on the DML operation (Triggering Event)

INSERT

UPDATE

DELETE

Therefore, there can be 12 types of triggers in Oracle based on the above classification

- BEFORE
 - INSERT - ROW LEVEL , STATEMENT LEVEL
 - UPDATE – ROW LEVEL , STATEMENT LEVEL
 - DELETE - ROW LEVEL , STATEMENT LEVEL
- AFTER
 - INSERT - ROW LEVEL , STATEMENT LEVEL
 - UPDATE – ROW LEVEL , STATEMENT LEVEL
 - DELETE - ROW LEVEL , STATEMENT LEVEL
- The 13th trigger is called INSTEAD OF TRIGGER

BEFORE TRIGGERS

- These triggers are fired before the triggering SQL statement (INSERT,UPDATE,DELETE) is executed. The execution of triggering SQL statement can be stopped depending on the various conditions to be fulfilled in BEFORE trigger.

AFTER TRIGGERS

- These triggers are fired after the triggering SQL statement (INSERT,UPDATE,DELETE) is executed. The triggering SQL statement is anyhow executed first followed by the code of trigger.

Statement level/Table level

- These triggers are fired for each DML operation being performed on a table.
- We cannot access the column values for records being inserted ,updated or deleted as it is meant for the table and not for the individual records.

Eg:

Consider a table tbl_audit with the foll. Fields:

Table_name varchar2(10)

Dml_operation varchar2(10)

Date_of_dml date

Create or replace trigger trig_emp_audit

After insert or update or delete on emp

Begin

if INSERTING then

 Insert into tbl_audit values('EMPLOYEE','INSERT',sysdate);

End if;

if UPDATING then

 Insert into tbl_audit values('EMPLOYEE',UPDATE',sysdate);

End if;

if DELETING then

 Insert into tbl_audit values('EMPLOYEE',DELETE',sysdate);

ROW LEVEL TRIGGERS

- These triggers are fired for each and every record which is inserted or deleted or updated from a table.
- In STATEMENT level triggers we cannot access the values of individual rows being inserted, updated or deleted

Note

- **:old.column_name and :new.column_name are reserved words of Oracle indicating the OLD and NEW values of columns.**
- **While INSERTING we can access only :new values**
- **While UPDATING we can access both :old and :new values**
- **While DELETING we can access only :old values**

- Create the 'product' table and 'product_price_history' table
- CREATE TABLE product (product_id number(5), product_name varchar2(32),supplier_name varchar2(32), unit_price number(7,2));
- CREATE TABLE product_price_history(product_id number(5) product_name varchar2(32),supplier_name varchar2(32),unit_price number(7,2));

- `select * from product;`

- o/p:

- `PRODUCT_ID   PRODUCT_NAME   SUPPLIER_NAME   UNIT_PRICE`

1	a	s	100
2	b	t	200
3	c	u	300
4	d	v	400

- CREATE or REPLACE TRIGGER
- BEFORE UPDATE OF unit_price
- ON product
- FOR EACH ROW
- BEGIN
- INSERT INTO product_price_history VALUES
- (:old.product_id,
- :old.product_name,
- :old.supplier_name,
- :old.unit_price);
- END;

- Lets update the price of a product.
- `UPDATE PRODUCT SET unit_price = 800 WHERE product_id = 1`
- Once the above update query is executed, the trigger fires and updates the 'product_price_history' table.

- `select * from product;`

- o/p:

- | PRODUCT_ID | PRODUCT_NAME | SUPPLIER_NAME | UNIT_PRICE |
|------------|--------------|---------------|------------|
|------------|--------------|---------------|------------|

- | | | | |
|---|---|---|-----|
| 1 | a | s | 800 |
| 2 | b | t | 200 |
| 3 | c | u | 300 |
| 4 | d | v | 400 |

- `select * from product_price_history ;`

- o/p:

PRODUCT_ID	PRODUCT_NAME	SUPPLIER_NAME	UNIT_PRICE
------------	--------------	---------------	------------

1	a	s	100
---	---	---	-----



Thank You

